Software Requirements Specification

for

CampusSphere – Campus Opportunity Hub

Version 1.0 approved

Prepared by

CampusSphere Project Team

Rajwardhan Singh - 24CSB1A52
Ramitha Ramesh - 24CSB1A53
Rayi Charan Harsha - 24CSB1A54

21/Sep/26

Table of Contents

Table of Contents **2**
Revision History **3**
1.Introduction **4**
1.1 Purpose 4
1.2 Document Conventions 4
1.3 Intended Audience and Reading Suggestions 4
1.4 Product Scope 4
1.5 References 4
2. Overall Description **5**
2.1 Product Perspective 5
2.2 Product Functions 5
2.3 User Classes and Characteristics 5
2.4 Operating Environment 6
2.5 Design and Implementation Constraints 6
2.6 User Documentation 6
2.7 Assumptions and Dependencies 6
3. External Interface Requirements **7**
3.1 User Interfaces 7
3.2 Hardware Interfaces 7
3.3 Software Interfaces 7
3.4 Communications Interfaces 7
4. System Features **8**
4.1 Account & Role Management 8
4.2 Opportunity Discovery & Search 8
4.3 AI-Powered Recommendation Engine 9
4.4 Application Submission & Tracking 9
4.5 Opportunity Posting & Approval Workflow 10
4.6 Notifications & Communication 11
4.7 Admin Moderation & Analytics 11
5. Other Nonfunctional Requirements **12**
5.1 Performance Requirements 12
5.2 Safety Requirements 12
5.3 Security Requirements 12
5.4 Software Quality Attributes 12
5.5 Business Rules 12
6. Other Requirements **14**
6.1 Database Requirements 14
6.2 Internationalization 14
6.3 Legal / Data Privacy Requirements 14
Appendix A: Glossary **15**
Appendix B: Analysis Models **15**
Appendix C: To Be Determined List **15**

Revision History

Name	Date	Reason For Changes	Version
Project Team	21/Sep/26	Initial approved draft	1.0

1. Introduction

1.1 Purpose

This document specifies the software requirements for CampusSphere, the Campus Opportunity Hub, Release 1.0. It covers the full system — the student/faculty/recruiter web client, the admin console, and the supporting backend services — as a single subsystem; there are no separate SRS documents for individual components at this release.

1.2 Document Conventions

Each functional requirement is uniquely identified with a sequence number in the form REQ-N. Requirement priority (High, Medium, Low) is stated once per System Feature in Section 4 and is inherited by all functional requirements listed under that feature unless a requirement states its own priority. The keyword “shall” indicates a mandatory requirement; “should” indicates a recommendation. “TBD” marks information not yet available; all TBD items are collected in Appendix C.

1.3 Intended Audience and Reading Suggestions

This document is intended for the student development team, and, secondarily, university stakeholders (Placement Cell, Faculty Coordinators) reviewing scope. Developers and testers should read Sections 2 through 5 in full; reviewers primarily interested in scope can read Section 1, Section 2, and the feature descriptions (4.x.1) without the detailed functional requirements or interface sections.

1.4 Product Scope

CampusSphere is a centralized web platform that lets students discover, apply for, and track campus opportunities (internships, hackathons, scholarships, research projects, competitions, workshops, certifications, and club recruitment), while faculty, recruiters, and student organizations post and manage those opportunities under an Admin-approved workflow. A recommendation engine, seeded by an onboarding survey and refined by ongoing student activity, personalizes what each student sees. Full business rationale, stakeholder analysis, and product positioning are provided in the companion CampusSphere Vision & Scope document; this SRS does not duplicate that content.

1.5 References

1. CampusSphere Vision & Scope Document, Version 1.0, Project Team.
2. CampusSphere Use Cases Document, Version 1.1, Project Team.
3. CampusSphere ProjectLibre Document — Work Breakdown Structure, Schedule, and Resource Plan, Project Team.

2. Overall Description

2.1 Product Perspective

CampusSphere is a new, independent web-based system; it is not a component of an existing university system, although it is expected to eventually integrate with the university's single sign-on and email systems. Students, Faculty, and Recruiters interact through a responsive web/mobile client; University Admins use a dedicated admin console. Both clients call a common REST API layer, which is backed by a PostgreSQL database, a recommendation engine, and a notification service, as shown below.

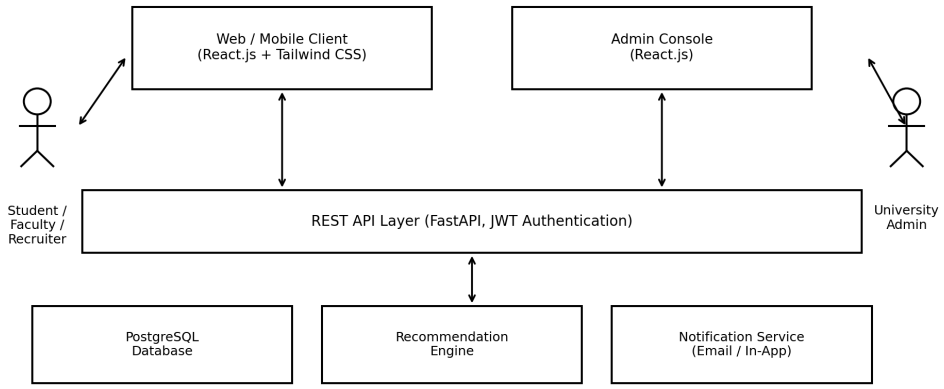


Fig 2.1 CampusSphere System Architecture

2.2 Product Functions

- Register, authenticate, and manage a role-based account (Student, Faculty, Recruiter, Admin).
- Complete an onboarding survey that seeds a skill/interest profile.
- Search, filter, and bookmark opportunities; receive personalized recommendations.
- Apply to an opportunity and track application status.
- Post an opportunity and route it through an Admin approval workflow.
- Receive deadline reminders and status-change notifications.
- Approve/reject postings, manage categories and tags, and view platform analytics (Admin).

2.3 User Classes and Characteristics

User Class	**Characteristics**
Student	Primary, high-frequency user; casual technical expertise; expects a simple, mobile-friendly experience; uses nearly all product functions except posting/approval.
Student Representative	Primary, moderate-frequency user drawn from the Student class; casual-to-moderate technical expertise; uses all standard Student functions plus relays peer feedback to Faculty/Admin and helps publicize postings within their class or department.
Faculty	Occasional user (periodic postings/reviews); moderate technical expertise; needs a low-friction posting and review workflow.
Recruiter	Occasional external user; posts internships/jobs and reviews applicants; needs account verification before posting.
Admin	Low-frequency but high-privilege user; approves organizations/postings, manages taxonomy, and monitors analytics; needs efficient bulk-review tools.

2.4 Operating Environment

The web/mobile client runs in any modern evergreen browser (Chrome, Edge, Firefox, Safari) on desktop or mobile, with no native app installation required. The backend runs as containerized FastAPI services on a Docker-capable Linux host, backed by a PostgreSQL database, deployed to a cloud provider (Render, Railway, or AWS).

2.5 Design and Implementation Constraints

- Fixed technology stack: React.js + Tailwind CSS + shadcn/ui (frontend), FastAPI (backend), PostgreSQL (database), JWT (authentication), Docker (deployment).
- The system must be delivered within the academic semester's project timeline (see the ProjectLibre schedule).
- Development is by a small student team with no dedicated budget beyond free-tier cloud services.
- Initial release supports English-language content only.

2.6 User Documentation

CampusSphere will ship with: a short online user guide covering account creation, the onboarding survey, and applying for opportunities; contextual tooltips and an in-app FAQ per role; and a README/deployment guide covering environment setup and Docker-based deployment for maintainers.

2.7 Assumptions and Dependencies

1. Users have reliable access to a smartphone or computer with a modern browser and internet access.
2. The university designates an Admin (or small team) to approve organizations and moderate postings.
3. Recruiters and Faculty complete a lightweight verification step before their postings go live.
4. Recommendation quality depends on Students completing the onboarding survey and remaining active; Students who do not will see only popularity-based recommendations.
5. Cloud hosting (web/database servers, backups) is available for the pilot deployment, and an optional third-party AI API may be used for recommendation generation.

3. External Interface Requirements

3.1 User Interfaces

The web/mobile client and admin console are built with React.js, Tailwind CSS, and shadcn/ui components, giving a consistent look and feel across both. Every screen shares a common header/navigation, a consistent set of buttons (Save, Cancel, Apply, Bookmark, Approve, Reject), and standardized inline error/validation messages. The UI targets WCAG 2.1 AA accessibility where practical and is responsive down to typical mobile screen widths. Detailed screen layouts are maintained separately in the team's UI wireframes/mockups (see the Product Overview diagram in the Vision & Scope document).

3.2 Hardware Interfaces

CampusSphere has no direct hardware interfaces. The client requires only a standard input device (touchscreen, keyboard/mouse) supported by the host browser; the backend requires a standard virtualized server (CPU, RAM, disk) capable of running Docker containers, with no specialized peripherals.

3.3 Software Interfaces

Interface	Purpose
PostgreSQL (v14+)	Primary relational data store for users, opportunities, applications, and related entities.
FastAPI (Python)	Backend REST API framework serving the web client and admin console.
JWT library	Issues and validates authentication tokens for all protected endpoints.
OpenAI API (optional)	Supplements the recommendation engine's matching/summarization logic, if enabled.
SMTP email service	Delivers deadline-reminder and status-change email notifications.
Docker	Packages backend services for consistent deployment across environments.
GitHub	Source control and CI for the application codebase.

3.4 Communications Interfaces

All client-server communication uses HTTPS/TLS-encrypted REST calls exchanging JSON payloads. Email notifications are sent via SMTP using a transactional email provider. No custom binary protocols are used. The API follows conventional REST semantics (GET/POST/PUT/DELETE) and returns standard HTTP status codes; authenticated requests carry a JWT bearer token in the Authorization header.

4. System Features

This section organizes the functional requirements by system feature, the major services CampusSphere provides. Each feature corresponds to one or more use cases in the companion Use Cases document.

4.1 Account & Role Management

4.1.1 Description and Priority

Allows a person to register, log in, and hold one of four roles (Student, Faculty, Recruiter, Admin) that determine which functions they can access, and lets a Student complete the onboarding survey that seeds their profile. Priority: High.

4.1.2 Stimulus/Response Sequences

1. User registers with an email address and password, or logs in with existing credentials.
2. System authenticates the credentials and issues a session (JWT).
3. System routes the user to the dashboard appropriate to their role.
4. A first-time Student is prompted to complete the onboarding survey (see Use Case 1).

4.1.3 Functional Requirements

REQ-1: The system shall allow a new user to register with a unique email address and a password.

REQ-2: The system shall authenticate login credentials using hashed password storage (e.g., bcrypt); passwords shall never be stored in plaintext.

REQ-3: The system shall issue a JWT upon successful authentication and require a valid JWT for all protected endpoints.

REQ-4: The system shall assign each account exactly one role (Student, Faculty, Recruiter, or Admin) and enforce role-based permissions on the server side, not only in the UI.

REQ-5: The system shall prompt a Student to complete the onboarding survey (skills, interests, academic program, goals) after first login.

REQ-6: The system shall allow a Student to save a partially completed survey and resume it at a later login.

REQ-7: The system shall allow a Student to edit their skill profile and replace their uploaded resume (PDF/DOCX, max 5 MB) at any time after onboarding.

4.2 Opportunity Discovery & Search

4.2.1 Description and Priority

Allows a Student to browse the catalog of open opportunities, either by manual search/filter or via personalized recommendations, and to bookmark opportunities of interest. Priority: High.

4.2.2 Stimulus/Response Sequences

1. Student enters a keyword and/or selects filters (category, tag, skill, deadline).
2. System returns matching, approved, open opportunities.
3. Student opens an opportunity's detail page and optionally bookmarks it.
4. System schedules deadline reminders for any bookmarked (or applied-to) opportunity.

4.2.3 Functional Requirements

REQ-8: The system shall allow a Student to search opportunities by keyword and filter by category, tag, skill, and deadline range.

REQ-9: The system shall return only approved, currently open opportunities in search and filter results.

REQ-10: The system shall return search and filter results within 2 seconds under normal load.

REQ-11: The system shall allow a Student to bookmark and remove bookmarks for any opportunity.

REQ-12: The system shall schedule reminder notifications at configurable intervals (default 7 days, 1 day, 3 hours) before the deadline of a bookmarked or applied-to opportunity.

REQ-13: The system shall cancel any pending reminders for an opportunity that is closed, withdrawn, or removed before its deadline.

4.3 AI-Powered Recommendation Engine

4.3.1 Description and Priority

Continuously generates and refines a ranked list of opportunity recommendations for each Student, combining their survey profile with ongoing behavior (applications, bookmarks, searches). Priority: Medium.

4.3.2 Stimulus/Response Sequences

1. Student views the “Recommended for You” section of their dashboard.
2. System scores all open opportunities against the Student's profile and recent activity.
3. System displays the ranked list with a brief reason for each recommendation.
4. System records the Student's response (viewed, bookmarked, applied, dismissed) as new feedback.

4.3.3 Functional Requirements

REQ-14: The system shall generate an initial set of recommendations from a Student's onboarding survey responses.

REQ-15: The system shall refine a Student's recommendations using their subsequent applications, bookmarks, and profile edits, at least once daily.

REQ-16: The system shall display, for each recommended opportunity, a short explanation of why it was recommended.

REQ-17: The system shall fall back to popularity-based (non-personalized) recommendations for a Student who has not completed the survey or has no activity history.

REQ-18: The system shall allow a Student to dismiss a recommendation as “not relevant,” which shall down-weight similar opportunities in future recommendation cycles.

REQ-19: The system shall only recommend opportunities that are approved and currently open.

4.4 Application Submission & Tracking

4.4.1 Description and Priority

Allows a Student to apply to an open opportunity using their existing profile and resume, and to track and, if needed, withdraw that application. Priority: High.

4.4.2 Stimulus/Response Sequences

1. Student selects “Apply” on an opportunity and reviews a pre-filled application form.
2. Student submits the application.

3. System creates the application record and notifies the Student and the Organizer.
4. Student later views “My Applications” to check status or withdraw the application.

4.4.3 Functional Requirements

REQ-20: The system shall pre-fill an application form with the Student's current profile and default resume, allowing edits before submission.

REQ-21: The system shall prevent a Student from submitting more than one application to the same opportunity.

REQ-22: The system shall reject an application submitted after the opportunity's stated deadline.

REQ-23: The system shall notify the Student and the opportunity's Organizer within 1 minute of a successful application submission.

REQ-24: The system shall display all of a Student's applications and their current status (Applied, Under Review, Shortlisted, Rejected, Selected) on a single dashboard.

REQ-25: The system shall allow a Student to withdraw an application that has not yet been marked “Selected,” subject to an explicit confirmation step.

4.5 Opportunity Posting & Approval Workflow

4.5.1 Description and Priority

Allows a Faculty member, Recruiter, or Club leader to submit a new opportunity, and routes it through Admin review before it becomes visible to Students. Priority: High.

4.5.2 Stimulus/Response Sequences

1. Organizer completes and submits the opportunity posting form.
2. System stores the posting with status “Pending Approval” and notifies the Admin.
3. Admin reviews the posting and approves, rejects, or requests changes.
4. System notifies the Organizer of the decision and, if approved, publishes the opportunity.

4.5.3 Functional Requirements

REQ-26: The system shall allow a verified Organizer (Faculty, Recruiter, or Club leader) to submit a new opportunity posting with title, description, category, tags, eligibility, and deadline.

REQ-27: The system shall prevent an unverified organization from submitting a posting and shall direct the Organizer to a verification request form.

REQ-28: The system shall route every new posting to “Pending Approval” status and notify the assigned Admin within 1 minute.

REQ-29: The system shall allow an Admin to approve, reject (with a stated reason), or request changes to a pending posting.

REQ-30: The system shall make an approved posting visible/searchable to Students and eligible for recommendation immediately upon approval.

REQ-31: The system shall allow an Organizer to save a posting as a draft, visible only to themselves, without triggering Admin review.

4.6 Notifications & Communication

4.6.1 Description and Priority

Delivers deadline reminders, application status-change alerts, and opportunity Q&A (comments) to the relevant users, with per-user control over notification channels. Priority: Medium.

4.6.2 Stimulus/Response Sequences

1. System sends an in-app and/or email notification when a tracked event occurs (deadline approaching, status change, new comment reply).
2. Student opens “Notification Settings” to choose channels (in-app, email) per notification type.
3. Student or Organizer posts a comment/question on an opportunity's detail page; the other party is notified and may reply.

4.6.3 Functional Requirements

REQ-32: The system shall send a notification for each deadline reminder interval and each application status change, via the channels the recipient has enabled.

REQ-33: The system shall allow a Student to enable or disable the email channel per notification type, but shall always keep in-app notifications enabled for application status changes.

REQ-34: The system shall allow a Student or Organizer to post a comment on an opportunity's detail page and notify the Organizer of a new Student question.

REQ-35: The system shall withhold a flagged or inappropriate comment from public view and route it to the Admin moderation queue.

4.7 Admin Moderation & Analytics

4.7.1 Description and Priority

Gives an Admin the tools to manage the platform's taxonomy (categories/tags), moderate reported content, and view platform-wide analytics on postings, applications, and engagement. Priority: High.

4.7.2 Stimulus/Response Sequences

1. Admin opens the Pending Approvals queue, Reported Content queue, or Analytics Dashboard.
2. System displays the relevant data (pending postings, flagged items, or usage metrics).
3. Admin takes an action (approve/reject, remove/dismiss, or export a report).

4.7.3 Functional Requirements

REQ-36: The system shall provide an Admin dashboard listing platform-wide metrics: postings, applications, active users, and recommendation engagement, filterable by date range.

REQ-37: The system shall allow an Admin to export analytics data as CSV.

REQ-38: The system shall allow an Admin to add, edit, merge, or remove categories and tags, with a warning if a tag being removed is in use by open opportunities.

REQ-39: The system shall allow an Admin to review flagged/reported content and remove it, warn the associated user, or dismiss the report.

REQ-40: The system shall log all Admin approval, rejection, and moderation actions for audit purposes.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- Search and filter results shall return within 2 seconds under normal pilot load (a few hundred concurrent users).
- Application submission shall complete within 3 seconds under normal load.
- The recommendation list shall load in under 2 seconds and refresh at least once daily.
- Notification delivery (in-app and email) shall have at least 99% reliability and typically complete within 1 minute of the triggering event.

5.2 Safety Requirements

CampusSphere presents no physical safety hazard. The primary “safety” concern is data integrity and loss prevention: no submitted application, bookmark, or posting shall be lost once accepted by the system, and the production database shall be backed up on a regular schedule.

5.3 Security Requirements

- All traffic shall be encrypted with HTTPS/TLS.
- Passwords shall be stored using a strong hashing algorithm (e.g., bcrypt); plaintext passwords shall never be stored or logged.
- All authenticated endpoints shall require a valid JWT; role permissions shall be enforced server-side.
- Admin actions (approvals, rejections, moderation) shall be logged for audit purposes (see REQ-40).
- Student personal data (profile, resume, application history) shall be accessible only to the Student themselves, the relevant Organizer for their own postings, and Admins.

5.4 Software Quality Attributes

Attribute	Target
Usability	A new Student shall be able to complete onboarding and submit a first application within 10 minutes without external help.
Availability	99% uptime during active recruitment periods (start of semester, major deadlines).
Reliability	No loss of submitted application, bookmark, or posting data once accepted.
Maintainability	Backend services are containerized (Docker) and documented (README, API reference) to ease onboarding of future maintainers.
Portability	The containerized backend shall be deployable to any Docker-capable cloud host (Render, Railway, or AWS) without code changes.
Scalability	The system shall support at least a few hundred concurrent users at pilot scale without redesign.

5.5 Business Rules

ID	Rule
BR-1	Every Student must have a profile before receiving personalized recommendations.
BR-2	Survey questions are configurable by Admin.
BR-3	Only approved, open opportunities may be recommended, searched, or filtered.
BR-4	A Student's activity data is used only to personalize their own recommendations.

BR-5	A Student may not apply twice to the same opportunity.
BR-6	Applications cannot be submitted after the stated deadline.
BR-7	Only verified organizations may post opportunities.
BR-8	All postings require Admin approval before becoming visible.
BR-9	Reminders are only sent for open opportunities.
BR-10	A Student may opt out of email reminders but not in-app notifications.
BR-11	Rejected postings must include a reason visible to the Organizer.
BR-12	Comments are subject to Admin moderation.
BR-13	A withdrawn application cannot be resubmitted to the same opportunity without Organizer approval.
BR-14	Category/tag names must be unique, case-insensitive.
BR-15	Each achievement-linked reward may be redeemed at most once per Student.

6. Other Requirements

6.1 Database Requirements

The system requires a normalized relational schema in PostgreSQL comprising, at minimum, the following tables: Users, Roles, Skills, UserSkills, Organizations, Opportunities, Categories, Applications, Bookmarks, Notifications, Achievements, Resumes, Recommendations, Comments, Tags, and OpportunityTags. Full schema design (columns, keys, relationships) is maintained in the ProjectLibre project plan's implementation tasks and the team's database design artifacts.

6.2 Internationalization

Release 1.0 supports English-language content only, per the Vision & Scope document's stated constraint. Multilingual support is out of scope for this release.

6.3 Legal / Data Privacy Requirements

The system shall be designed with the university's data-privacy obligations for student data in mind, limiting access to personal data (profile, resume, application history) to the Student themselves, relevant Organizers, and Admins, as stated in Section 5.3.

Appendix A: Glossary

Term	Definition
CampusSphere	The Campus Opportunity Hub platform described by this document.
Opportunity	An internship, hackathon, scholarship, research project, workshop, certification, or club recruitment posted on CampusSphere.
Organizer	Collective term for Faculty, Recruiter, or Student Club Leadership roles that post opportunities.
JWT	JSON Web Token, used for authenticating users.
RBAC	Role-Based Access Control.
REQ-N	Unique identifier assigned to each functional requirement in Section 4.
TBD	To Be Determined — a placeholder for information not yet available; see Appendix C.
SRS	Software Requirements Specification (this document).

Appendix B: Analysis Models

Supporting analysis models are maintained in companion documents rather than duplicated here:

- System architecture diagram: Figure 2.1 of this document.
- Use case diagrams and detailed use case descriptions: CampusSphere Use Cases Document.
- Entity-relationship / database schema (16 tables listed in Section 6.1): CampusSphere ProjectLibre Document and team database design artifacts.
- Work breakdown structure and project schedule: CampusSphere ProjectLibre Document.

Appendix C: To Be Determined List

1. Whether minor edits to an already-approved posting require re-approval (currently assumed no) — Section 4.5, to be confirmed with the Placement Cell.

2. Whether approval authority should be delegated to department-level Faculty Coordinators to reduce Admin bottleneck — Section 4.7, under discussion.

3. Whether recommendation “match score” should be shown numerically or only qualitatively — Section 4.3, to be decided during UI design.

4. Whether deadline-reminder intervals should be configurable per Student rather than fixed platform-wide — Section 4.2, candidate for a future release.

5. Who supplies and funds vouchers for the achievements/rewards feature — to be confirmed with University Administration before that feature is built.